

EcommerceWebAR

Tienda de figuras coleccionables con Realidad Aumentada

Guia de instalacion local

De cero a la pagina corriendo en tu PC

Que vas a lograr al terminar esta guia:

Clonar el repositorio, configurar la base de datos y los archivos de entorno, levantar el backend y el frontend, y dejar la tienda funcionando en tu navegador con catalogo, AR, carrito, ventas y panel de administracion.

Stack: React 19 + Vite 8 • Express 5 • Supabase (PostgreSQL)

Documento generado para el equipo de desarrollo

Contents

1	Requisitos previos	2
1.1	Programas obligatorios	2
1.2	Verifica que todo este instalado	2
1.3	Editor recomendado	2
2	Clonar el repositorio	2
3	Instalar las dependencias	3
4	Configurar la base de datos (Supabase)	3
4.1	Crear el proyecto en Supabase	3
4.2	Crear las tablas (schema)	3
4.3	Sembrar roles y permisos (datos de prueba)	4
4.4	Aplicar la migracion de ventas	4
4.5	Crear el bucket de almacenamiento	4
4.6	Obtener las llaves de la API	4
5	Configurar las variables de entorno	4
5.1	Backend: <code>server/.env</code>	5
5.2	Frontend: <code>client/.env</code>	5
6	Crear el usuario administrador	5
7	Levantar la aplicacion	6
7.1	Opcion A: un solo comando (recomendada)	6
7.2	Opcion B: dos terminales (manual)	6
7.3	Abrir la pagina	6
8	Probar que todo funciona	7
9	Solucion de problemas comunes	7
10	Resumen rapido (checklist)	8

1. Requisitos previos

Antes de empezar necesitas tener instalado lo siguiente en tu PC. Esta guia asume Windows, macOS o Linux indistintamente; donde haya diferencias se indica.

1.1 Programas obligatorios

1. **Node.js version 20.6 o superior.** Es imprescindible esta version o mas nueva, porque el backend usa `process.loadEnvFile()`, una funcion que solo existe a partir de Node 20.6. Lo recomendable es instalar la version LTS mas reciente (22.x).
2. **npm.** Viene incluido con Node.js, no se instala aparte.
3. **Git.** Para clonar el repositorio.
4. **Una cuenta de Supabase** (gratuita) en <https://supabase.com>. La base de datos y el almacenamiento de imagenes/modelos viven ahi.

1.2 Verifica que todo este instalado

Abre una terminal y ejecuta:

```
node --version      # debe mostrar v20.6.0 o superior
npm --version       # cualquier version reciente
git --version        # cualquier version
```

IMPORTANTE

Si `node -version` muestra algo menor que v20.6.0, la pagina NO va a arrancar. Actualiza Node antes de continuar. Puedes descargarlo de <https://nodejs.org> o usar un gestor de versiones como `npm`.

1.3 Editor recomendado

Cualquiera sirve, pero se recomienda **Visual Studio Code** por su integracion con JavaScript, React y las extensiones de ESLint/Prettier que el proyecto ya trae configuradas.

2. Clonar el repositorio

Ubica en tu terminal la carpeta donde quieras guardar el proyecto y ejecuta:

```
git clone https://github.com/Faraysito/EcommerceWebAr.git
cd EcommerceWebAr
```

Quedaras dentro de la carpeta del proyecto. La estructura principal es:

```
EcommerceWebAr/
|-- client/      # Frontend (React + Vite)
|-- server/      # Backend (Express + Supabase)
|-- database/    # Scripts SQL (schema y datos de prueba)
|-- package.json # Monorepo: npm workspaces
```

► Nota

El proyecto es un **monorepo con npm workspaces**. Eso significa que **client** y **server** son dos sub-proyectos, pero se manejan desde la raíz con un solo **npm install**. No tienes que instalar dependencias por separado en cada carpeta.

3. Instalar las dependencias

Desde la carpeta raíz del proyecto (EcommerceWebAr/), ejecuta:

```
npm install
```

Esto instala las dependencias del monorepo, del frontend y del backend de una sola vez (gracias a los workspaces). Puede tardar un par de minutos la primera vez.

► Nota

Si en el futuro necesitas instalar una librería solo para el frontend o solo para el backend, usa el parametro **-workspace**:

```
npm install <paquete> --workspace=server # solo backend  
npm install <paquete> --workspace=client  # solo frontend
```

4. Configurar la base de datos (Supabase)

Esta es la parte mas larga, pero solo se hace una vez. Vamos por pasos.

4.1 Crear el proyecto en Supabase

1. Entra a <https://supabase.com> e inicia sesión.
2. Crea un nuevo proyecto (botón *New project*). Ponle el nombre que quieras (por ejemplo **figura-ar**) y elige una contraseña para la base de datos (guardala, aunque no la usaras directamente en esta guía).
3. Espera a que el proyecto termine de aprovisionarse (1-2 minutos).

4.2 Crear las tablas (schema)

1. En el panel de Supabase, abre el **SQL Editor** (icono en la barra lateral).
2. Abre el archivo **database/schema.sql** del proyecto, copia **todo** su contenido y pégalo en el editor.
3. Presiona *Run*. Esto crea todas las tablas: **user**, **role**, **permission**, **category**, **product**, **image**, **model**, **customer**, **sale**, **sale_product**, **offer**, etc.

IMPORTANTE

La tabla **user** es una palabra reservada en PostgreSQL y por eso en el schema va siempre entre comillas dobles: **"user"**. El bloque final del schema que activa Row Level Security (RLS) ya usa las comillas correctas. No edites esas líneas: si las tocas y quitas las comillas, el script fallara.

4.3 Sembrar roles y permisos (datos de prueba)

Repite el proceso del SQL Editor con el archivo `database/test-values.sql`: copia su contenido, pegalo y ejecuta *Run*. Esto crea el rol `admin` y le asigna los ~40 permisos del sistema.

4.4 Aplicar la migracion de ventas

IMPORTANTE

Este paso es **obligatorio** si tu copia del proyecto incluye el sistema de carrito y ventas. Corre en el SQL Editor el archivo `database/migration-sales.sql`. Anade la columna `customer.name` y unos indices. No recrea tablas, asi que es seguro ejecutarlo.

4.5 Crear el bucket de almacenamiento

Las imagenes y los modelos 3D (`.glb`) se guardan en Supabase Storage.

1. En el panel de Supabase, abre la seccion **Storage**.
2. Crea un nuevo bucket. Ponle un nombre (por ejemplo `products`). Recuerda este nombre: lo necesitaras en el archivo de entorno del backend.
3. **Marca el bucket como publico** (*Public bucket*). Esto es imprescindible: si el bucket es privado, las imagenes de los productos apareceran rotas en la tienda.

► Nota

Si una imagen aparece rota en el catalogo, la causa numero uno es que el bucket no quedo marcado como publico. La numero dos es que al crear el producto en el panel no se le asigno ninguna imagen.

4.6 Obtener las llaves de la API

El backend se conecta a Supabase con dos datos que tienes que copiar:

1. En el panel, ve a **Project Settings** → **API**.
2. Copia la **Project URL** (algo como `https://xxxxx.supabase.co`).
3. Copia la llave **service_role** (la secreta, empieza por `sb_secret_...` o es un JWT largo). El backend la usa para saltarse las reglas de RLS y operar sobre las tablas.

IMPORTANTE

La llave `service_role` es **secreta**. Nunca la subas a GitHub ni la pegues en el codigo del frontend. Solo va en el archivo `.env` del backend, que esta ignorado por Git.

5. Configurar las variables de entorno

El proyecto necesita dos archivos `.env` (uno por sub-proyecto). Ambos tienen un archivo de ejemplo `.env-example` que puedes copiar.

5.1 Backend: server/.env

Crea un archivo llamado `.env` dentro de la carpeta `server/` (al lado de `.env-example`) con este contenido, rellenando tus datos de Supabase:

```
# Opcionales
NODE_ENV="development"
PORT=3000

# Con cookies de sesion, el origen NO puede ser "*".
# Debe ser la lista del/los origen(es) del frontend.
ALLOWED_ORIGINS=["http://localhost:5173"]

# Obligatorios (de tu proyecto Supabase)
SUPABASE_URL="https://xxxxx.supabase.co"
SUPABASE_KEY="tu_llave_service_role_secreta"
SUPABASE_BUCKET="products"
JWT_SECRET="una_cadena_secreta_de_minimo_32_caracteres"
```

SUPABASE_URL

La Project URL que copiaste.

SUPABASE_KEY

La llave `service_role`.

SUPABASE_BUCKET

El nombre exacto del bucket que creaste.

JWT_SECRET

Cualquier cadena larga y aleatoria, minimo 32 caracteres. Sirve para firmar los tokens de sesion. Puedes inventarla o generar una con `openssl rand -hex 32`.

ALLOWED_ORIGINS

La direccion del frontend. En local es `http://localhost:5173`. Tiene que ir como un array JSON valido.

IMPORTANTE

ALLOWED_ORIGINS no puede ser `"*"` porque la sesion usa cookies con **credentials**. Los navegadores rechazan el comodin en ese caso. Debe ser la lista explicita de origenes, por ejemplo `["http://localhost:5173"]`.

5.2 Frontend: client/.env

Crea un archivo `.env` dentro de la carpeta `client/` con una sola linea, que apunta al backend:

```
VITE_API_URL="http://localhost:3000"
```

6. Crear el usuario administrador

El panel de administracion (`/admin`) necesita un usuario superadmin. El proyecto trae un script que lo crea por ti. Desde la carpeta `server/`, ejecuta:

```
cd server
npm run seed:admin
```

Esto crea un usuario con las credenciales por defecto:

Email: admin@figura-ar.cl **Contrasena:** admin12345

► Nota

Puedes cambiar esas credenciales definiendo las variables `ADMIN_EMAIL` y `ADMIN_PASSWORD` en el `.env` del backend antes de correr el script. Si el usuario ya existe, el script no hace nada.

IMPORTANTE

Para que este script funcione, primero tienen que existir el rol `admin` (lo creaste con `test-values.sql`) y las variables `SUPABASE_URL` y `SUPABASE_KEY` en el `.env`. Si no, el script te avisara con un error.

7. Levantar la aplicacion

Hay dos formas de arrancar: la rapida (un solo comando) o la manual (dos terminales). Cualquiera sirve.

7.1 Opcion A: un solo comando (recomendada)

Desde la **carpeta raiz** del proyecto:

```
npm run dev:both
```

Esto levanta el backend y el frontend al mismo tiempo (usa `concurrently`). Veras en la misma terminal los mensajes de los dos servidores.

7.2 Opcion B: dos terminales (manual)

Si prefieres ver cada servidor por separado, abre **dos terminales**.

En la primera, el backend:

```
cd server
npm run dev
```

Deberias ver: **Server running on port 3000.**

En la segunda, el frontend:

```
cd client
npm run dev
```

Vite te mostrara una URL, normalmente `http://localhost:5173`.

7.3 Abrir la pagina

Abre tu navegador en:

`http://localhost:5173`

✓ **Si todo salio bien:** veras la tienda con el header naranja, el buscador y el catalogo. Si ya creaste productos desde el panel, apareceran aqui. El panel de administracion esta en `http://localhost:5173/admin`.

8. Probar que todo funciona

Sigue este recorrido para confirmar que cada parte responde:

1. **Panel admin.** Entra a `/admin`, inicia sesion con `admin@figura-ar.cl / admin12345`.
2. **Crear contenido.** En el panel: crea una categoria, sube una imagen (y opcionalmente un modelo `.glb`) en la pestana Archivos, y luego crea un producto asignandole esa categoria e imagen.
3. **Ver la tienda.** Vuelve a `http://localhost:5173`: el producto creado debe aparecer en el catalogo.
4. **Buscador.** Escribe en el buscador del header; el catalogo debe filtrarse por nombre.
5. **Carrito y compra.** Agrega el producto al carrito, abrelo, pulsa comprar, crea una cuenta de cliente y confirma el pedido. El stock debe bajar.
6. **Pedidos.** Desde el menu de cuenta (arriba a la derecha), entra a *Mis pedidos*: debe aparecer la compra.
7. **Usuarios.** En `/admin`, pestana Usuarios: lista, crea y elimina usuarios del panel.

9. Solucion de problemas comunes

La pagina no arranca / error de `loadEnvFile`

Tu version de Node es menor que 20.6. Actualiza Node y vuelve a intentar.

Error de CORS o la sesion no se mantiene

Revisa que `ALLOWED_ORIGINS` en `server/.env` sea exactamente `["http://localhost:5173"]` (array JSON, sin comodin `*`) y que el frontend este corriendo en ese puerto.

Las imagenes de los productos aparecen rotas

Dos causas: (1) el bucket de Storage no quedo marcado como *publico*, o (2) al crear el producto no le asignaste ninguna imagen. Revisa ambas.

El backend no conecta a Supabase

Verifica `SUPABASE_URL` y `SUPABASE_KEY` en `server/.env`. La llave debe ser la `service_role`, no la `anon`.

`npm run seed:admin` falla diciendo que no existe el rol admin

Te falto correr `database/test-values.sql` en el SQL Editor de Supabase.

El puerto 3000 o 5173 esta ocupado

Cierra el proceso que lo usa, o cambia `PORT` en `server/.env` (y actualiza `VITE_API_URL` en `client/.env` para que coincida).

10. Resumen rapido (checklist)

Para alguien que ya conoce el flujo, estos son todos los pasos condensados:

1. `git clone` del repo y entrar a la carpeta.
2. `npm install` desde la raiz.
3. En Supabase: crear proyecto, correr `schema.sql`, `test-values.sql` y `migration-sales.sql` en el SQL Editor.
4. En Supabase: crear bucket `products` y marcarlo **publico**.
5. Copiar Project URL + llave `service_role`.
6. Crear `server/.env` (URL, KEY, BUCKET, JWT_SECRET, ALLOWED_ORIGINS).
7. Crear `client/.env` (VITE_API_URL).
8. `cd server && npm run seed:admin`.
9. `npm run dev:both` desde la raiz.
10. Abrir `http://localhost:5173`.

Fin de la guia. Si la pagina carga en el navegador y puedes iniciar sesion en el panel, !todo esta funcionando!